OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Data Handling using Pandas: Series & DataFrames

Data Structures, Creation Patterns, Core Attributes, Index Alignment, Vectorized Math & Slicing Invariants

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B2: Topic 1 (Class 12)	Data Structures in Pandas: Series (1D homogeneous labeled array) and DataFrame (2D heterogeneous tabular data structure).	Series size immutability vs DataFrame size mutability, Series `ndim=1` and `shape=(n,)` vs DataFrame `ndim=2` and `shape=(r, c)`.
Section B2: Topic 1 (Class 12)	Series Creation & Attributes: Creation from ndarray, dictionary, scalar, list. Attributes: `values`, `index`, `dtype`, `size`, `empty`, `head()`, `tail()`.	Dict keys becoming index labels, missing keys in explicit index producing `NaN`, upcasting integers to `float64`, scalar replication.
Section B2: Topic 1 (Class 12)	DataFrame Operations & Slicing: Creation from dict of lists / dict of Series, attributes (`axes`, `columns`, `dtypes`, `T`), selection and slicing.	Dict of lists requiring equal lengths vs dict of Series allowing variable lengths, `loc` inclusive label slice vs `iloc` exclusive integer slice.

EXAM PATTERN

Pandas questions in CUET Computer Science / Informatics Practices focus extensively on data structures, slicing boundaries, and alignment:

1. **Mutability Invariant:** A Pandas Series is **Value Mutable** but **Size Immutable** (you can update element values, but you cannot insert or delete elements). A DataFrame is **both Value Mutable and Size Mutable** (you can dynamically add/drop rows and columns).
2. **The Fatal Slicing Boundary Trap:** Positional slicing (`s[0:3]` or `df.iloc[0:3]`) is a standard half-open interval that **excludes** the stop index (indices 0, 1, 2). Label-based slicing (`s['a':'c']` or `df.loc['a':'c']`) is a closed interval that **INCLUDES** the stop label (`'c'`)!
3. **Dict of Lists vs Dict of Series Constructor Trap:** When constructing a DataFrame from a Dictionary of Lists, all lists **must be of identical length** (mismatched lengths throw a fatal `ValueError`). In contrast, a Dictionary of Series allows **different lengths**; missing indices automatically fill with `NaN`!
4. **Automatic Label Alignment:** When performing arithmetic operations between two Series (`s1 + s2`), Pandas aligns data by **Index Label**, NOT positional order! Unmatched labels produce `NaN` (which upcasts the Series to `float64`).
5. **Single vs Double Bracket Column Selection:** `df['Score']` returns a 1D Pandas **Series**. `df[['Score']]` returns a 2D Pandas **DataFrame** containing a single column!
6. **Shape Tuple Formatting:** For a Series with 5 elements, `s.shape` returns strictly `(5,)` (a 1-tuple), **never** `(5, 1)`! For a DataFrame with 5 rows and 3 columns, `df.shape` returns `(5, 3)`.

Python Data Analysis Ecosystem & Pandas Architecture

Modern data-driven applications, scientific computing, and artificial intelligence workflows in Python rely on an integrated hierarchy of open-source analytical libraries:

1. **NumPy (Numerical Python):** Supplies high-performance, contiguous, N-dimensional homogeneous numerical arrays (`ndarray`) with C-optimized vectorized linear algebra routines, broadcasting, and Fourier transforms.
2. **Pandas (Python Data Analysis Library):** Built directly on top of NumPy, Pandas introduces two expressive, high-level data structures—**Series** (1D) and **DataFrame** (2D)—specifically engineered for handling structured, labeled, heterogeneous tabular data, relational joins, time-series analysis, and missing value imputation.
3. **Matplotlib & Seaborn:** High-level 2D data visualization plotting libraries providing publication-quality line charts, bar graphs, histograms, and scatter plots.

The Two Foundational Pandas Data Structures

Pandas abstracts data processing into two primary data structures:

Feature	Pandas Series	Pandas DataFrame
Dimensionality	1-Dimensional (1D) array	2-Dimensional (2D) tabular grid / spreadsheet
Data Homogeneity	Homogeneous (All elements share a single `dtype`)	Heterogeneous (Different columns can have different `dtypes`)
Index / Axes	Single axis: Row `Index` (Axis 0)	Two axes: Row `Index` (Axis 0) and `Columns` (Axis 1)
Value Mutability	Mutable (Values can be modified in-place)	Mutable (Values can be modified in-place)
Size Mutability	**Immutable** (Cannot add/delete elements in-place)	**Mutable** (Columns and rows can be inserted/deleted)
Memory Model	Wraps a 1D NumPy array with an index	Collection of aligned Series sharing a common row index

Pandas Series: Architecture, Creation Modes & Type Coercion

A **Pandas Series** is a one-dimensional labeled array capable of holding data of any type (integers, floating-point numbers, strings, Python objects). The axis labels are collectively termed the **Index**.

1. Syntax of the Series Constructor

```
import pandas as pd
s = pd.Series(data=None, index=None, dtype=None, name=None, copy=False)
```

- If the `index` parameter is omitted, Pandas automatically assigns a default zero-indexed integer range: `0, 1, 2, ..., n-1`.
- If an explicit `index` list is supplied, it must contain unique or non-unique hashable labels of the exact same length as `data`.

2. Creation from Python Lists and 1D NumPy Arrays

Passing a standard Python list or 1D NumPy array initializes the Series values:

```
import pandas as pd
import numpy as np

# 1. Default integer index (0 to 3)
s1 = pd.Series([10, 20, 30, 40])
# Output:
# 0    10
# 1    20
# 2    30
# 3    40
# dtype: int64

# 2. Custom explicit string labels
s2 = pd.Series(np.array([100, 200, 300]), index=['Jan', 'Feb', 'Mar'])
```

- **The Length Matching Invariant:** When specifying an explicit `index` list alongside a list or ndarray, the length of the `index` list **must strictly equal** the length of the data array. If lengths differ, Pandas throws a fatal `ValueError: Length of passed values is X, index implies Y`!

3. Type Coercion & Data Homogeneity in Series

Because a Series is internally backed by a homogeneous NumPy array, all elements must share a common data type:

- **Integers and Floats:** If a list contains mixed integers and floating-point numbers `[10, 20.5, 30]`, Pandas upcasts all integers to floats, resulting in `dtype: float64`.
- **Numbers and Strings:** If a list contains mixed numbers and strings `[100, 'Apple', 200]`, Pandas coerces the entire Series to generic Python pointers, resulting in `dtype: object`.
- **Booleans:** A list of `[True, False, True]` produces `dtype: bool`.

4. Creation from Python Dictionaries (CUET High-Yield)

When a Python dictionary is passed to `pd.Series(dict_obj)`:

- The dictionary **Keys** automatically become the Series **Index Labels**.
- The dictionary **Values** become the Series data elements.
- The order of elements strictly preserves the insertion order of dictionary keys.

```
data_dict = {'Math': 95, 'Physics': 88, 'Chemistry': 92}
s_marks = pd.Series(data_dict)
# Output:
# Math         95
# Physics      88
# Chemistry    92
# dtype: int64
```

- **Custom Index with Dictionary (The Missing Key Invariant):** If an explicit `index` list is supplied alongside a dictionary, Pandas filters the dictionary to match the specified index labels:
  - If a label exists in the dictionary keys, its value is extracted.
  - If an index label does **not** exist as a key in the dictionary, Pandas inserts `NaN` (**Not a Number**) for that label!

```
s_custom = pd.Series(data_dict, index=['Math', 'Physics', 'Biology'])
# Output:
# Math       95.0
# Physics    88.0
# Biology     NaN   <- Key 'Biology' was absent in data_dict!
# dtype: float64    <- Automatically upcasted from int64 to float64!
```

- **The Float64 Upcasting Invariant:** In Python/NumPy, integer arrays cannot represent missing values. Therefore, whenever `NaN` is introduced into an integer Series, Pandas **automatically upcasts the entire Series `dtype` from `int64` to `float64`**!

5. Creation from Scalar Values

If `data` is a single scalar constant (e.g. integer, float, string) and an `index` list is provided, Pandas replicates the scalar value across every single specified index label:

```
s_scalar = pd.Series(100, index=['A', 'B', 'C', 'D'])
# Output:
# A    100
# B    100
# C    100
# D    100
# dtype: int64
```

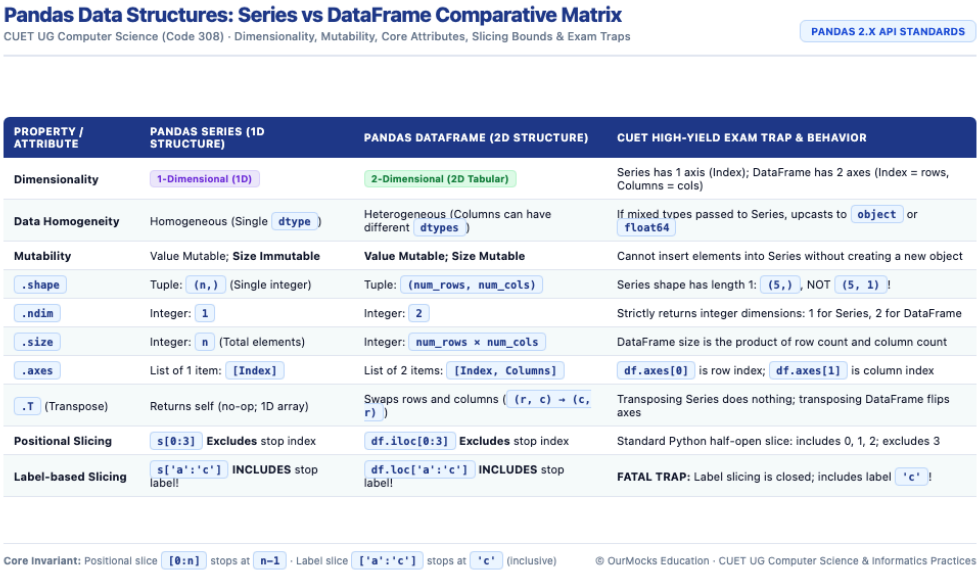
SERIES CREATION & NAN UPCASTING ANALYSIS

Problem: Consider the following Python dictionary: `sales = {'Q1': 5000, 'Q2': 7500, 'Q3': 6000}`. An analyst executes: `s = pd.Series(sales, index=['Q2', 'Q3', 'Q4'])`.

1. What is the output of `s['Q4']`?
2. What is the value of `s['Q2']`?
3. What is the resulting `s.dtype`?

Solution:

- **Part 1:** The key `'Q4'` is absent from the input dictionary `sales`. Pandas inserts `NaN` (Not a Number) for `'Q4'`.
- **Part 2:** The key `'Q2'` exists with value `7500`. Its value is preserved as `7500.0`.
- **Part 3:** Because `NaN` is inherently a floating-point IEEE 754 value, Pandas upcasts the entire integer sequence to `float64`.



Pandas Data Structures: Series vs DataFrame Comparative Matrix

CUET UG Computer Science (Code 308) · Dimensionality, Mutability, Core Attributes, Slicing Bounds & Exam Traps

PANDAS 2.X API STANDARDS

PROPERTY / ATTRIBUTE	PANDAS SERIES (1D STRUCTURE)	PANDAS DATAFRAME (2D STRUCTURE)	CUET HIGH-YIELD EXAM TRAP & BEHAVIOR
Dimensionality	1-Dimensional (1D)	2-Dimensional (2D Tabular)	Series has 1 axis (Index); DataFrame has 2 axes (Index = rows, Columns = cols)
Data Homogeneity	Homogeneous (Single `dtype`)	Heterogeneous (Columns can have different `dtypes`)	If mixed types passed to Series, upcasts to `object` or `float64`
Mutability	Value Mutable; **Size Immutable**	**Value Mutable; Size Mutable**	Cannot insert elements into Series without creating a new object
`.shape`	Tuple: `(n,)` (Single integer)	Tuple: `(num_rows, num_cols)`	Series shape has length 1: `(5,)`, NOT `(5, 1)`!
`.ndim`	Integer: `1`	Integer: `2`	Strictly returns integer dimensions: 1 for Series, 2 for DataFrame
`.size`	Integer: `n` (Total elements)	Integer: `num_rows × num_cols`	DataFrame size is the product of row count and column count
`.axes`	List of 1 item: `[Index]`	List of 2 items: `[Index, Columns]`	`df.axes[0]` is row index; `df.axes[1]` is column index
`.T` (Transpose)	Returns self (no-op; 1D array)	Swaps rows and columns (`(r, c) → (c, r)`)	Transposing Series does nothing; transposing DataFrame flips axes
Positional Slicing	`s[0:3]` **Excludes** stop index	`df.iloc[0:3]` **Excludes** stop index	Standard Python half-open slice: includes 0, 1, 2; excludes 3
Label-based Slicing	`s['a':'c']` **INCLUDES** stop label!	`df.loc['a':'c']` **INCLUDES** stop label!	**FATAL TRAP:** Label slicing is closed; includes label `'c'`!

Core Invariant: Positional slice `[0:n]` stops at `n-1` · Label slice `['a':'c']` stops at `'c'` (inclusive)

© OurMocks Education · CUET UG Computer Science & Informatics Practices

Figure 1: Pandas Data Structures Architecture Matrix: Series vs DataFrame Dimensionality, Attributes & Slicing Invariants

Pandas Series Attributes & Vectorized Operations

Pandas Series provide rich inspection attributes and support element-wise vectorized arithmetic.

1. Core Attributes of a Series Object

Assuming `s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])`:

- `s.values`: Returns the underlying data elements as a 1D NumPy `ndarray` (`array([10, 20, 30])`).
- `s.index`: Returns the index object containing axis labels (`Index(['a', 'b', 'c'], dtype='object')`).
- `s.dtype`: Returns the data type of the underlying elements (e.g. `dtype('int64')` or `dtype('float64')`).
- `s.shape`: Returns a tuple representing the dimensionality. For a Series with n elements, `s.shape` is strictly `(n,)` (a 1-tuple, e.g. `(3,)`), **never** `(3, 1)`!
- `s.ndim`: Returns the number of dimensions as an integer: strictly `1` for all Series.
- `s.size`: Returns the total number of elements as an integer: strictly `n` (e.g. `3`).
- `s.nbytes`: Returns the total memory in bytes consumed by the data array ($3 \times 8 = 24$ bytes for `int64`).
- `s.empty`: Returns a boolean: `True` if the Series contains zero elements; `False` otherwise.
- `s.hasnans`: Returns `True` if the Series contains one or more `NaN` values.

2. Indexing and Slicing in Pandas Series

Elements can be retrieved using either positional indices or label names:

- **Positional Indexing:** Zero-based integer offsets (`s[0]`, `s[1]`, `s[-1]`). Negative offsets count from the end.
- **Label-based Indexing:** Named keys (`s['a']`, `s['b']`).
- **The Crucial Positional vs Label Slicing Invariant (CUET 1 Scoring Trap):**
  - **Positional Slicing (`s[start:stop]`):** Standard Python half-open interval. **Excludes the stop index!** `s[0:2]` retrieves elements at index `0` and `1`; it **excludes** index `2`.
  - **Label-based Slicing (`s['start_label':'stop_label']`):** Closed interval. **INCLUDES the stop label!** `s['a':'c']` retrieves elements `'a'`, `'b'`, and `'c'`; label `'c'` is fully included!
- **Slicing with Step (`s[start:stop:step]`):** `s[::2]` retrieves every second element. `s[::-1]` reverses the entire Series in-place.

3. Vectorized Arithmetic & Automatic Label Alignment

When performing arithmetic operations (`+`, `-`, `*`, `/`, `//`, `%`) between two Series:

- Operations are **vectorized**—executed element-by-element in C without writing explicit Python `for` loops.
- **Automatic Label Alignment:** Pandas matches values by **Index Label**, not positional sequence!

```
s1 = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
s2 = pd.Series([1, 2, 3], index=['b', 'c', 'd'])
s3 = s1 + s2
# Output:
# a     NaN   (Missing in s2)
# b    21.0   (20 + 1 matched at label 'b')
# c    32.0   (30 + 2 matched at label 'c')
```

```
# d     NaN   (Missing in s1)
# dtype: float64
```

- **NaN Invariant:** Any arithmetic operation with `NaN` yields `NaN` ($10 + \text{NaN} = \text{NaN}$).
- **The `add()` Method with `fill_value`:** To prevent `NaN` propagation, use `s1.add(s2, fill_value=0)`. Any missing label in one Series is replaced with `0` before addition, preserving values.

4. Series Inspection & Statistical Aggregation Methods

- `s.head(n)`: Returns the first n elements (default $n = 5$).
- `s.tail(n)`: Returns the last n elements (default $n = 5$).
- `s.count()`: Returns the number of **non-null** (valid) elements. **Ignores NaN!**
- `s.sum()`, `s.mean()`, `s.median()`, `s.min()`, `s.max()`: Standard statistical summaries (all ignore `NaN` by default).

SERIES MATHEMATICAL ALIGNMENT NUMERICAL ANALYSIS

Problem: Consider two Series: `p = pd.Series([100, 200, 300], index=['Pen', 'Pencil', 'Eraser'])` `q = pd.Series([50, 100], index=['Pencil', 'Ruler'])` Calculate the output of `p + q` and `p.add(q, fill_value=0)`.
Solution:

- **Standard Addition (`p + q`):**
  - `'Eraser'`: Absent in `q` $\rightarrow$ `NaN`
  - `'Pen'`: Absent in `q` $\rightarrow$ `NaN`
  - `'Pencil'`: $200 + 50 = 250.0$
  - `'Ruler'`: Absent in `p` $\rightarrow$ `NaN`

  Result: `'Eraser': NaN, 'Pen': NaN, 'Pencil': 250.0, 'Ruler': NaN`.
- **Addition with `fill_value=0` (`p.add(q, fill_value=0)`):**
  - `'Eraser'`: $300 + 0 = 300.0$
  - `'Pen'`: $100 + 0 = 100.0$
  - `'Pencil'`: $200 + 50 = 250.0$
  - `'Ruler'`: $0 + 100 = 100.0$

  Result: All 4 items have valid numbers; zero `NaN` values!

Pandas DataFrame: Architecture & Constructor Creation Patterns

A **Pandas DataFrame** is a two-dimensional, size-mutable, heterogeneous tabular data structure with labeled axes (rows and columns). It conceptually resembles a SQL table or a spreadsheet.

1. DataFrame Coordinate System: Axes

- **Axis 0 (Rows / Index):** Represents records (observations) running vertically down rows. Operations along `axis=0` compute column-wise statistics.
- **Axis 1 (Columns):** Represents features (variables) running horizontally across columns. Operations along `axis=1` compute row-wise statistics.

2. Creation from a Dictionary of Lists or 1D ndarrays

In this pattern, dictionary keys become DataFrame **Column Names**, and the lists become the column values:

```
import pandas as pd

student_data = {
    'RollNo': [101, 102, 103],
    'Name': ['Aarav', 'Bhavna', 'Chetan'],
    'Marks': [88, 94, 76]
}
df_students = pd.DataFrame(student_data, index=['s1', 's2', 's3'])
```

- **The Strict Equal Length Rule:** All lists within the dictionary **must be of identical length**! If `'RollNo'` has 3 items and `'Marks'` has 2 items, Pandas raises `ValueError: All arrays must be of the same length`!

3. Creation from a Dictionary of Series (CUET High-Yield)

When dictionary values are Pandas Series:

- Column names are derived from dictionary keys.
- Row index labels are formed by the **Union of all individual Series indices**!
- **Unequal Lengths Allowed:** Unlike lists, Series **do not need to be of equal length**. Any missing row label in a Series is automatically padded with `NaN`!

```
eng = pd.Series([85, 90], index=['R1', 'R2'])
math = pd.Series([92, 88, 95], index=['R1', 'R2', 'R3'])
df_marks = pd.DataFrame({'English': eng, 'Math': math})
# Output:
```

```
#     English  Math
# R1     85.0    92
# R2     90.0    88
# R3      NaN    95   <- Automatically filled with NaN!
```

4. Creation from a 2D NumPy ndarray

A 2D matrix can be passed directly with optional custom `index` and `columns` lists:

```
matrix = np.array([[10, 20, 30], [40, 50, 60]])
df_mat = pd.DataFrame(matrix, index=['Row1', 'Row2'], columns=['ColA', 'ColB', 'ColC'])
# Shape: (2, 3)
```

5. Creation from a List of Dictionaries

Each dictionary in the list represents a single row; dictionary keys become column names:

```
rows = [{'Name': 'Deepak', 'Age': 17}, {'Name': 'Esha', 'Age': 18, 'Grade': 'A'}]
df_people = pd.DataFrame(rows)
# 'Grade' for Deepak automatically becomes NaN!
```

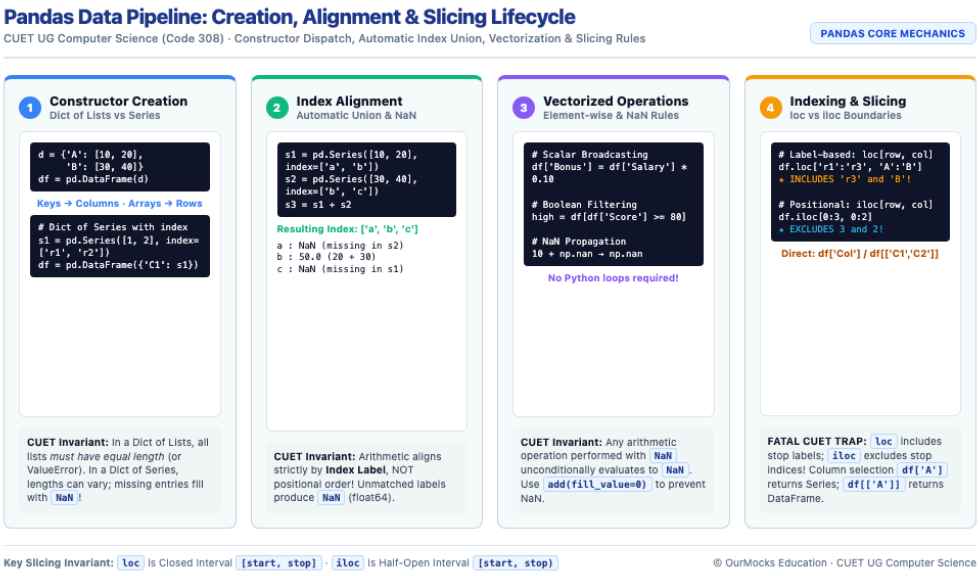


Figure 2: Pandas Data Pipeline: Constructor Dispatch, Automatic Index Union, Vectorization & Slicing Rules

DataFrame Core Attributes & Inspection Methods

Assuming `df = pd.DataFrame({'A': [10, 20], 'B': [30, 40], 'C': [50, 60]}, index=['R1', 'R2'])`:

- `df.shape`: Returns tuple `(num_rows, num_columns)`. For `df`, returns strictly `(2, 3)`.
- `df.ndim`: Returns integer dimensionality: strictly `2` for all DataFrames.
- `df.size`: Returns total element count: `num_rows * num_columns` ($2 \times 3 = 6$).
- `df.index`: Returns row labels object (`Index(['R1', 'R2'], dtype='object')`).
- `df.columns`: Returns column names object (`Index(['A', 'B', 'C'], dtype='object')`).
- `df.axes`: Returns a list of the two axes: `[df.index, df.columns]`. `df.axes[0]` is rows; `df.axes[1]` is columns.
- `df.dtypes`: Returns a Series displaying the data type of each column.
- `df.values`: Returns the raw data as a 2D NumPy `ndarray`.
- `df.empty`: Returns boolean: `True` if DataFrame has zero rows or zero columns.
- `df.T` (Transpose): Swaps rows and columns. Shape transforms from `(r, c)` to `(c, r)`.
- `df.head(n)`: Returns the first n rows. Default value when n is omitted is strictly **5**.
- `df.tail(n)`: Returns the last n rows. Default value when n is omitted is strictly **5**.

DataFrame Selection, Column Manipulation & Boolean Filtering

Pandas provides intuitive syntax for extracting, inserting, deleting, and filtering tabular records.

1. Column Selection: Single vs Double Brackets

- **Single Bracket (`df['ColumnName']`):** Extracts a single column as a **1D Pandas Series**.

- **Double Brackets (`df[['ColA', 'ColB']]`):** Extracts multiple columns (or a single column formatted as a table) as a **2D Pandas DataFrame**.

```
col_series = df['A']      # type: pd.Series, shape: (2,)
col_df = df[['A']]        # type: pd.DataFrame, shape: (2, 1)
```

2. Adding and Deleting Columns

- **Adding a Column via Assignment:**
  - Assigning a scalar value broadcasts to all rows: `df['Discount'] = 0.05`.
  - Assigning a calculated expression: `df['Total'] = df['Price'] * df['Quantity']`.
- **Deleting a Column:**
  - `del df['Discount']`: Deletes the column in-place permanently.
  - `df.pop('Discount')`: Deletes the column in-place and returns the deleted Series.
  - `df.drop('Discount', axis=1)`: Returns a new DataFrame with the column removed (leaves original `df` unchanged unless `inplace=True`).

3. Row Selection & Slicing: `loc` vs `iloc`

- `df.loc[row_label, col_label]`: Label-based indexing. Both row and column slices are **inclusive** of the stop label!
- `df.iloc[row_pos, col_pos]`: Integer-based positional indexing. Both row and column slices are **exclusive** of the stop index!

```
# Select rows 'R1' through 'R2', columns 'A' through 'B' (INCLUSIVE)
sub1 = df.loc['R1':'R2', 'A':'B']

# Select first 2 rows (0, 1) and first 2 columns (0, 1) (EXCLUSIVE of 2)
sub2 = df.iloc[0:2, 0:2]
```

- **Scalar Accessors:** `df.at['R1', 'A']` (fast label scalar access) and `df.iat[0, 0]` (fast integer scalar access).

4. Boolean Indexing (Filtering Rows by Condition)

Rows can be filtered by passing boolean condition expressions inside brackets:

```
# Filter rows where column 'Score' exceeds 80
passed_students = df[df['Score'] > 80]

# Combining multiple conditions using bitwise operators (& for AND, | for OR)
elite = df[(df['Score'] > 80) & (df['Attendance'] >= 90)]
```

- **The Parentheses Invariant:** In Python/Pandas, bitwise operators (`&`, `|`) have higher precedence than comparison operators (`>`, `<`). Therefore, **each individual condition must be enclosed in parentheses** `(df['A'] > 50) & (df['B'] < 100)`, or Python raises a fatal `TypeError`!

DATAFRAME ATTRIBUTE & SLICING OUTPUT TRACING

Problem: Consider the DataFrame `df` created by:

```
import pandas as pd
data = {'P': [10, 20, 30], 'Q': [40, 50, 60], 'R': [70, 80, 90]}
df = pd.DataFrame(data, index=['X', 'Y', 'Z'])
```

1. What is the output of `df.shape` and `df.size`?
2. What is the output of `df.iloc[0:2, 1:3]`?
3. What is the output of `df.loc['X':'Y', 'Q':'R']`?

Solution:

- **Part 1:** `df` has 3 rows and 3 columns:
$$\text{df.shape} = (3, 3) \quad \text{and} \quad \text{df.size} = 3 \times 3 = 9$$
- **Part 2:** `df.iloc[0:2, 1:3]` uses positional indexing:
  - Rows: index 0 (`'X'`) and index 1 (`'Y'`). (Index 2 `'Z'` is excluded).
  - Columns: position 1 (`'Q'`) and position 2 (`'R'`). (Position 3 is excluded).

  Output is a 2×2 DataFrame containing rows `['X', 'Y']` and columns `['Q', 'R']`.
- **Part 3:** `df.loc['X':'Y', 'Q':'R']` uses label indexing:
  - Rows `'X'` through `'Y'` (both included).
  - Columns `'Q'` through `'R'` (both included).

  The output is identical to Part 2: rows `['X', 'Y']` and columns `['Q', 'R']`!

Missing Data Representation, NaN Propagation & Imputation Mechanics

In data analytics, missing or unrecorded observations are an inevitable reality. Pandas represents missing numerical data using `np.nan` (Not a Number), conforming to the IEEE 754 floating-point standard. In string or object series, `None` is automatically converted into `np.nan`.

1. Detection of Missing Values

Pandas offers four complementary boolean mask methods:

- `s.isna()` or `s.isnull()`: Returns a boolean structure of identical shape, where cells containing `NaN` evaluate to `True`, and valid cells evaluate to `False`.
- `s.notna()` or `s.notnull()`: The exact logical inverse; returns `True` for non-missing elements.
- **Counting Null Values:** In CUET questions, finding missing value frequencies is evaluated using aggregate chaining:
  - `df.isnull().sum()`: Computes the count of `NaN` values **per column** (aggregates along Axis 0).
  - `df.isnull().sum().sum()`: Returns the total scalar count of `NaN` values across the entire DataFrame grid.

2. Dropping Missing Data: `.dropna()`

The `.dropna()` method filters out records containing null values:

```
# Drops any row where AT LEAST ONE column is NaN (default: axis=0, how='any')
df_clean_rows = df.dropna()

# Drops a column only if ALL elements in that column are NaN
df_clean_cols = df.dropna(axis=1, how='all')

# Retains rows having at least 3 non-null values
df_thresh = df.dropna(thresh=3)
```

3. Imputation & Filling: `.fillna()`

Rather than discarding records, missing entries can be filled with constants or imputed statistics:

- Constant fill: `df.fillna(0)` replaces all missing cells with zero.
- Statistical imputation: `df['Score'].fillna(df['Score'].mean())` replaces missing scores with the column mean.
- Directional propagation:
  - Forward Fill: `df.fillna(method='ffill')` carries the last observed non-null value forward downwards.
  - Backward Fill: `df.fillna(method='bfill')` carries the subsequent valid value backwards upwards.

CUET HIGH-YIELD TRAP: IN-PLACE MUTATION

By default, `.dropna()`, `.fillna()`, and `.drop()` do **not** modify the original DataFrame; they return a **new modified copy**. To mutate the original DataFrame in-place, you must explicitly pass `inplace=True`, or reassign the result: `df = df.fillna(0)`.

DataFrame Binary Arithmetic & Alignment Across Axes

When applying arithmetic operations (`+`, `-`, `*`, `/`) between two DataFrames, or between a DataFrame and a Series:

- Alignment occurs across **both** row index labels and column labels simultaneously (outer join).
- Where labels intersect in both operands, arithmetic executes element-wise.
- Where a label is unique to one operand, the resulting intersection evaluates to `NaN`.

Method Equivalents with `fill_value`

To guard against `NaN` cascades during addition or subtraction, Pandas exposes explicit arithmetic methods:

- `.add()` (`+`), `.sub()` (`-`), `.mul()` (`*`), `.div()` or `.divide()` (`/`)
- `.radd()`, `.rsub()`, `.rmul()`, `.rdiv()` (reversed operands: `df.rdiv(100)` evaluates `100 / df`)

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]}, index=['R1', 'R2'])
df2 = pd.DataFrame({'B': [10, 20], 'C': [30, 40]}, index=['R2', 'R3'])

# Direct addition generates extensive NaNs:
res_op = df1 + df2

# Method addition with fill_value=0 treats missing cells as 0 before adding:
res_method = df1.add(df2, fill_value=0)
```

Iterating Over Pandas DataFrames

While vectorized operations are vastly preferred for computational performance, certain procedural algorithms require iterating across tabular records.

1. Column Iteration (`df.items()` / `df.iteritems()`)

Iterates over `(column_name, Series)` pairs. Each column is extracted as a Series along with its label:

```
for col_name, col_data in df.items():
    print(f"Column: {col_name}, Mean: {col_data.mean()}")
```

2. Row Iteration via `.iterrows()`

Iterates over `(index_label, Series)` pairs. Each row is converted into a 1D Pandas Series:

```
for idx, row in df.iterrows():
    print(f"Row {idx}: Eng={row['Eng']}, Math={row['Math']}")
```

- **Note:** Because each row is cast into a Series, data types may be upcast to a common type (e.g., if a row contains integers and floats, the row Series is upcast to `float64`).

3. Row Iteration via `.itertuples()`

Iterates over rows as lightweight Python `namedtuple` objects:

- Significantly faster than `.iterrows()` because it avoids the overhead of instantiating a Series for every row.
- Preserves native data types across fields without type coercion.
- Access fields via attribute notation: `row.Eng`, `row.Math`, or tuple indices: `row[1]`.

File Ingestion & Persistence: CSV Operations in Pandas

The comma-separated values (CSV) format represents the universal standard for flat tabular data interchange. Pandas provides robust, highly optimized I/O routines: `pd.read_csv()` and `df.to_csv()`.

1. Reading CSV Files (`pd.read_csv`)

```
df = pd.read_csv(
    filepath_or_buffer='students.csv',
    sep=',',                # Delimiter character (default: ',')
    header=0,               # Row number containing column names (default: 0)
    index_col=0,            # Column to use as row labels (default: None -> RangeIndex)
    usecols=['Name', 'Score'] # Load only a subset of columns
)
```

- If a CSV has no column names in its first row, pass `header=None` and provide a list of names via `names=['Col1', 'Col2']`.

2. Exporting to CSV Files (`df.to_csv`)

```
df.to_csv('output.csv', index=False)
```

THE CRITICAL `INDEX=FALSE` CUET INVARIANT

When exporting a DataFrame using `df.to_csv()`, Pandas by default writes the row index labels as the first column in the file. If you subsequently read the CSV back using `pd.read_csv()` without `index_col=0`, an unwanted extra column named `Unnamed: 0` is created! Passing `index=False` prevents writing row labels to disk.

COMPLEX DATAFRAME CONSTRUCTION & MISSING VALUE ARITHMETIC

Problem: Consider the two Series definitions below:

```
import pandas as pd
s1 = pd.Series([10, 20, 30], index=['Delhi', 'Mumbai', 'Chennai'])
s2 = pd.Series([5, 15, 25], index=['Mumbai', 'Chennai', 'Kolkata'])
df = pd.DataFrame({'Sales_A': s1, 'Sales_B': s2})
```

1. What is the value of `df.shape` and `df.size`?
2. What is the output of `df.loc['Delhi', 'Sales_B']`?
3. What is the resulting output of `df['Sales_A'].add(df['Sales_B'], fill_value=0)['Kolkata']`?

Solution:

- **Part 1:** The row index of `df` is the union of indices: `['Delhi', 'Mumbai', 'Chennai', 'Kolkata']` (4 rows). There are 2 columns (`'Sales_A'` and `'Sales_B'`).

$$\text{df.shape} = (4, 2) \quad \text{and} \quad \text{df.size} = 4 \times 2 = 8$$

- **Part 2:** `'Delhi'` does not exist in `s2`, so Pandas places `NaN` in column `'Sales_B'`.

$$\text{df.loc['Delhi', 'Sales_B']} = \text{NaN}$$

- **Part 3:** For `'Kolkata'`, `'Sales_A'` is missing (filled with 0), while `'Sales_B'` has value 25.

$$0 + 25 = 25.0$$

Comprehensive CUET Code Traces & Analytical Walkthroughs

Solving numerical and code-tracing problems in CUET Computer Science requires systematic, step-by-step state evaluation. Below are high-yield worked traces reflecting the exact pattern of questions appearing in past papers.

MULTI-STEP DATAFRAME MUTATION, SLICING & DIMENSIONAL ANALYSIS

Problem: Consider the following script executed in an interactive Python session:

```
import pandas as pd
import numpy as np

data = {
    'RollNo': [101, 102, 103, 104],
    'Name': ['Kavita', 'Rohan', 'Sneha', 'Tariq'],
    'Marks': [88, 92, np.nan, 79],
    'Grade': ['B', 'A', 'F', 'C']
}
df = pd.DataFrame(data, index=['R1', 'R2', 'R3', 'R4'])

# Step 1: Add column 'FeePaid'
df['FeePaid'] = True

# Step 2: Delete column 'Grade' using pop
g_col = df.pop('Grade')

# Step 3: Extract subset using loc
sub = df.loc['R2':'R3', ['Name', 'Marks']]
```

1. What is the value of `type(g_col)` and `df.shape` after Step 2?
2. What is the exact output of `sub`?
3. What is the scalar result of `df['Marks'].isnull().sum()`?

Solution:

- **Part 1:** The `.pop()` method removes a column in-place and returns it as a 1D Series:
$$\text{type(g_col)} =< \text{class 'pandas.core.series.Series'} >$$
Initially, `df` had 4 rows and 4 columns. Step 1 added `'FeePaid'` (making 5 columns). Step 2 removed `'Grade'` (reducing to 4 columns). The number of rows remained 4.
$$\text{df.shape} = (4, 4)$$
- **Part 2:** In `.loc[]`, row slices are **inclusive** of the stop label `'R3'`. The extracted rows are `'R2'` and `'R3'`. The specified columns are `'Name'` and `'Marks'`.
$$\text{Output: } \begin{bmatrix} \text{R2} & \text{Rohan} & 92.0 \\ \text{R3} & \text{Sneha} & \text{NaN} \end{bmatrix}$$
- **Part 3:** Column `'Marks'` contains values `[88.0, 92.0, NaN, 79.0]`. Exactly one entry is `np.nan`.
$$\text{df['Marks'].isnull().sum()} = 1$$

SERIES ARITHMETIC WITH MISALIGNED INDICES & FILL VALUES

Problem: Two schools record participation counts across four sports activities:

```
import pandas as pd
s_east = pd.Series([45, 60, 30], index=['Cricket', 'Football', 'Tennis'])
s_west = pd.Series([55, 40, 25], index=['Football', 'Basketball', 'Cricket'])

res_direct = s_east + s_west
res_filled = s_east.add(s_west, fill_value=0)
```

1. What is the value of `res_direct['Basketball']` and `res_direct['Cricket']`?
2. How many elements in `res_direct` evaluate to `NaN`?
3. What is the value of `res_filled['Tennis']` and `res_filled['Football']`?

Solution:

- **Part 1:** `'Basketball'` is missing from `s_east`, so direct addition produces `NaN`. For `'Cricket'`, both Series contain values: $45 + 25 = 70.0$.
$$\text{res_direct['Basketball']} = \text{NaN}, \quad \text{res_direct['Cricket']} = 70.0$$
- **Part 2:** The union of indices is `['Basketball', 'Cricket', 'Football', 'Tennis']` (4 items). Only `'Cricket'` and `'Football'` exist in both Series. The remaining two (`'Basketball'` and `'Tennis'`) evaluate to `NaN`. Thus, exactly **2** elements are `NaN`.
- **Part 3:** In `res_filled`, `'Tennis'` is missing from `s_west`, so `fill_value=0` is used: $30 + 0 = 30.0$. For `'Football'`, $60 + 55 = 115.0$.
$$\text{res_filled['Tennis']} = 30.0, \quad \text{res_filled['Football']} = 115.0$$

Master Synthesis Mindmap: Pandas Series & DataFrames Foundations

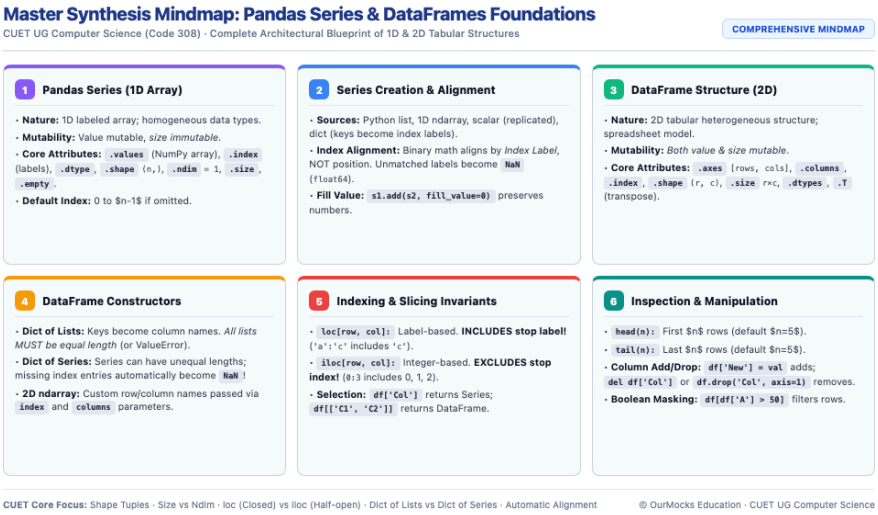


Figure 3: Master Synthesis Mindmap: Pandas Data Structures, Constructor Alignment, Attributes & Slicing Rules

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

RAPID REVISION CHEATSHEET: PANDAS SERIES & DATAFRAMES FOUNDATIONS

- **Data Structures:**
  - **Series (1D):** Homogeneous, Value Mutable, **Size Immutable**. Axis 0 (`index`). Shape: `(n,)`.
  - **DataFrame (2D):** Heterogeneous tabular, **Value & Size Mutable**. Axes: Axis 0 (`index`, rows) and Axis 1 (`columns`). Shape: `(r, c)`.
- **Series Creation:**
  - From list/ndarray: `pd.Series([1, 2, 3])`. Default index: `0` to `n-1`. Lengths must match if custom index passed.
  - From dict: Keys become index labels. Extra index labels fill with `NaN` and upcast to `float64`.
  - From scalar: Scalar replicated for all labels in index.
- **Series Attributes:**
  - `s.values` (NumPy array), `s.index`, `s.dtype`, `s.shape` (`(n,)`), `s.ndim` (1), `s.size` (n), `s.empty`, `s.hasnans`.
- **DataFrame Creation:**
  - Dict of Lists: Keys = columns. **All lists must be equal length** (or `ValueError`).
  - Dict of Series: Keys = columns. Different lengths allowed; unmatched rows fill with `NaN`.
  - 2D ndarray: Direct matrix; custom `index` and `columns`.
- **DataFrame Attributes:**
  - `df.shape` (`(r, c)`), `df.size` (`r * c`), `df.ndim` (2), `df.index`, `df.columns`, `df.axes` (`[index, columns]`), `df.dtypes`, `df.T` (transpose: `(c, r)`).
- **Indexing & Slicing:**
  - `loc[row, col]`: Label-based. **INCLUDES stop label** (`'a':'c'` includes `'c'`).
  - `iloc[row, col]`: Integer-based. **EXCLUDES stop index** (`0:3` includes 0, 1, 2).
  - `df['Col']`: Returns Series. `df[['Col']]`: Returns DataFrame.
  - `df.head(n)` / `df.tail(n)`: Default $n = 5$ rows.
- **Vectorized Math & Alignment:**
  - Operates by **Index Label**, NOT position! Unmatched labels produce `NaN`.
  - `NaN` propagation: any math with `NaN` yields `NaN`. Use `df.add(df2, fill_value=0)`.

THE SEVEN FATAL CUET TRAPS IN PANDAS DATA HANDLING

1. **The Slicing Stop Boundary Trap:** `df.iloc[0:3]` stops at index `2` (excludes 3). `df.loc['a':'c']` stops at label `'c'` (**INCLUDES `'c'`**)! Failing to include the stop label in `loc` is the 1 CUET scoring error.
2. **The Series Shape Tuple Trap:** For a Series with 4 elements, `s.shape` is strictly `(4,)` (a 1-tuple with a trailing comma), **never** `(4, 1)` or integer `4`!
3. **The Dict of Lists Equal Length Trap:** Passing lists of different lengths to `pd.DataFrame({'A': [1, 2], 'B': [3, 4, 5]})` throws a fatal `ValueError`! Unequal lengths are only permitted when passing a Dictionary of **Series**.
4. **The Integer Upcasting to float64 Trap:** When a dictionary missing a key is converted to a Series with that key in the index, `NaN` is inserted. Because `NaN` is a float, Pandas upcasts the entire Series from `int64` to `float64`.
5. **The Single vs Double Bracket Selection Trap:** `df['Marks']` returns a 1D `Series`. `df[['Marks']]` returns a 2D `DataFrame`. Knowing the return type is critical for method chaining.
6. **The Boolean Filtering Parentheses Trap:** When filtering with multiple conditions, writing `df[df['A'] > 10 & df['B'] < 20]` causes a fatal `TypeError` due to operator precedence. You **must** write `df[(df['A'] > 10) & (df['B'] < 20)]`.
7. **The Size Mutability Trap:** A Series is **Size Immutable** (elements cannot be appended or dropped in-place without creating a new object). A DataFrame is **Size Mutable** (`df['New'] = val` inserts a column in-place).

CUET Focused Multiple Choice Questions (Part 1)

Q1. Which of the following correctly describes the mutability properties of Pandas Series and DataFrame data structures?

(A) Series is Value Mutable and Size Immutable; DataFrame is both Value and Size Mutable
(B) Series is both Value and Size Mutable; DataFrame is Value Mutable and Size Immutable
(C) Both Series and DataFrame are Value Immutable and Size Mutable
(D) Series is Value Immutable; DataFrame is both Value and Size Immutable

Sol. Q1. In Pandas, a Series is 1D with fixed size (Value Mutable, Size Immutable; values can change but length cannot change in-place). A DataFrame is 2D with dynamic dimensions (both Value Mutable and Size Mutable; columns and rows can be inserted or deleted).
Answer: (A)

Q2. Consider the code snippet:

```
import pandas as pd
d = {'A': 10, 'B': 20, 'C': 30}
s = pd.Series(d, index=['B', 'C', 'D'])
```

What is the resulting output of `s['D']` and the `s.dtype`?

(A) `0` and `int64`
(B) `NaN` and `float64`
(C) `None` and `object`
(D) `KeyError` exception thrown

Sol. Q2. The key `'D'` is absent from dictionary `d`. Pandas inserts `NaN` for missing index labels. Because `NaN` is a floating-point IEEE 754 value, Pandas upcasts the entire integer sequence to `float64`.
Answer: (B)

Q3. What is the return value of the attribute `s.shape` for a Pandas Series object `s` containing exactly 5 elements?

(A) `5`
(B) `(5, 1)`
(C) `(5,)`
(D) `[5]`

Sol. Q3. A Series is a one-dimensional array. Its `.shape` attribute returns a 1-tuple containing the number of elements with a trailing comma: `(5,)`. In contrast, a 2D DataFrame with 5 rows and 1 column would have shape `(5, 1)`.
Answer: (C)

Q4. Given a DataFrame `df` with row index labels `['r1', 'r2', 'r3', 'r4']` and column names `['A', 'B', 'C']`, which expression extracts rows `'r1'` through `'r3'` inclusive of `'r3'`?

(A) `df.iloc['r1':'r3']`
(B) `df.iloc[0:2]`
(C) `df['r1':'r3']`
(D) `df.loc['r1':'r3']`

Sol. Q4. The `.loc` accessor performs label-based indexing where slices are closed intervals that include the endpoint. Therefore, `df.loc['r1':'r3']` includes rows `'r1'`, `'r2'`, and `'r3'`. In contrast, `.iloc` requires integer positions and excludes stop indices.
Answer: (D)

Q5. What happens when an analyst attempts to create a DataFrame from a dictionary of lists where the lists have different lengths: `pd.DataFrame({'A': [10, 20], 'B': [30, 40, 50]})`?

(A) A `ValueError` is raised stating that all arrays must be of the same length
(B) The shorter list `'A'` is automatically padded with `NaN`
(C) The shorter list `'A'` repeats its first element to match length
(D) The longer list `'B'` is truncated to length 2

Sol. Q5. When constructing a DataFrame from a Dictionary of Lists or 1D ndarrays, Python requires all arrays to possess identical lengths. If lengths differ, Pandas raises `ValueError: All arrays must be of the same length`. (Padding with `NaN` occurs only when passing a Dictionary of **Series**).
Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. Consider two Pandas Series:

```
s1 = pd.Series([10, 20], index=['a', 'b'])
s2 = pd.Series([30, 40], index=['b', 'c'])
res = s1 + s2
```

What is the resulting output of `res`?

(A) `a: 40, b: 60, c: NaN`
(B) `a: NaN, b: 50.0, c: NaN`
(C) `a: 10, b: 50, c: 40`
(D) `b: 50.0`

Sol. Q6. Pandas aligns arithmetic operations strictly by Index Label. Label `'b'` exists in both Series, so $20 + 30 = 50.0$. Labels `'a'` and `'c'` exist in only one Series, so their results evaluate to `NaN`. The resulting Series has labels `'a'`, `'b'`, `'c'` with dtype `float64`.

Answer: (B)

Q7. What is the data type and dimensionality of the object returned by executing `df[['Marks']]` on a DataFrame `df`?

(A) A 1-dimensional Pandas Series
(B) A 1-dimensional NumPy ndarray
(C) A 2-dimensional Pandas DataFrame with 1 column
(D) A Python list of marks

Sol. Q7. Using double brackets `df[['Marks']]` passes a list of column names, which strictly returns a 2-dimensional `DataFrame` object with shape `(n, 1)`. In contrast, single bracket `df['Marks']` returns a 1-dimensional `Series`.

Answer: (C)

Q8. If a DataFrame `df` has 6 rows and 4 columns, what is the output of `df.size` and `df.ndim` respectively?

(A) `size = 10, ndim = 2`
(B) `size = 24, ndim = (6, 4)`
(C) `size = 6, ndim = 1`
(D) `size = 24, ndim = 2`

Sol. Q8. The `.size` attribute returns the total number of elements: num_rows $\times$ num_columns $= 6 \times 4 = 24$. The `.ndim` attribute returns the number of axes / dimensions: strictly integer `2` for any DataFrame.

Answer: (D)

Q9. Which Pandas inspection method returns the first 5 rows of a DataFrame by default when no argument is supplied?

(A) `df.head()`
(B) `df.top()`
(C) `df.first()`
(D) `df.tail()`

Sol. Q9. `df.head()` returns the first n rows of a DataFrame or Series, where $n = 5$ by default. Conversely, `df.tail()` returns the last 5 rows by default.

Answer: (A)

Q10. What is the output of executing `df.T.shape` on a DataFrame `df` that originally has `df.shape = (8, 3)`?

(A) `(8, 3)`
(B) `(3, 8)`
(C) `24`
(D) `(3,)`

Sol. Q10. The `.T` attribute transposes the DataFrame, swapping row index and column axes. If the original shape is $(r, c) = (8, 3)$, the transposed DataFrame has shape $(c, r) = (3, 8)$.

Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. Consider a Series `s = pd.Series([100, 200, 300, 400], index=['p', 'q', 'r', 's'])`. What does the positional slice `s[1:3]` return?

(A) Elements at `'p'`, `'q'`, and `'r'`
(B) Elements at `'q'`, `'r'`, and `'s'`
(C) Elements at `'q'` (200) and `'r'` (300)
(D) Elements at `'q'` (200) only

Sol. Q11. Positional slicing uses standard Python zero-based indexing where the start offset is included and the stop offset is excluded. Offset 1 corresponds to `'q'` (200), offset 2 corresponds to `'r'` (300), and offset 3 is excluded.
Answer: (C)

Q12. How can an analyst permanently delete a column named `'Bonus'` from a DataFrame `df` without creating a new DataFrame object?

(A) `df.drop('Bonus')`
(B) `df.remove('Bonus')`
(C) `df.pop('Bonus', axis=1)`
(D) `del df['Bonus']`

Sol. Q12. The Python `del` statement (`del df['Bonus']`) permanently removes the column in-place from the DataFrame. (Alternatively, `df.drop('Bonus', axis=1, inplace=True)` or `df.pop('Bonus')` can also delete in-place).
Answer: (D)

Q13. What is the return value of `df.axes` for a DataFrame `df` with row index `['r1', 'r2']` and columns `['c1', 'c2', 'c3']`?

(A) A list of two items: `[Index(['r1', 'r2']), Index(['c1', 'c2', 'c3'])]`
(B) A tuple: `(2, 3)`
(C) An integer: `2`
(D) A dictionary mapping rows to columns

Sol. Q13. The `.axes` attribute returns a Python list containing the labels of both axes: `[row_index, column_index]`. For this DataFrame, it returns `[Index(['r1', 'r2']), Index(['c1', 'c2', 'c3'])]`.
Answer: (A)

Q14. Which code correctly filters a DataFrame `df` to extract records where column `'Age'` is greater than or equal to 18 AND column `'City'` is `'Delhi'`?

(A) `df[df['Age'] >= 18 and df['City'] == 'Delhi']`
(B) `df[(df['Age'] >= 18) & (df['City'] == 'Delhi')]`
(C) `df[df['Age'] >= 18 && df['City'] == 'Delhi']`
(D) `df[(df['Age'] >= 18) and (df['City'] == 'Delhi')]`

Sol. Q14. In Pandas, vectorized boolean filtering mandates the bitwise operator `&` (not logical `and`) and requires each individual comparison clause to be enclosed in parentheses to prevent operator precedence errors.
Answer: (B)

Q15. What is the result of executing `pd.Series(25, index=['a', 'b', 'c'])`?

(A) A Series with 1 element `25` at index `'a'`
(B) A TypeError stating that scalar cannot be converted to Series
(C) A Series where scalar `25` is replicated across all index labels `'a'`, `'b'`, and `'c'`
(D) A Series with `25` at index `0` and `NaN` at `'a'`, `'b'`, `'c'`

Sol. Q15. When a single scalar value is supplied with an `index` argument, Pandas broadcasts the scalar value to match the specified index length, creating a Series with `25` at every index label: `'a'`, `'b'`, and `'c'`.
Answer: (C)